

M1_Machine_Learning_Introducció_i_Regressió_Lineal

December 12, 2025

0.1 Machine Learning

El machine learning va suposar un canvi paradigmàtic en l'Intel·ligència Artificial passant d'un enfocament basat en regles rígides i programació explícita a sistemes que aprenen patrons directament de les dades, millorant el rendiment mitjançant l'experiència sense necessitar instruccions predefinides.

Abans del machine learning, els sistemes d'IA es basaven en models rígids amb regles codificades manualment per experts, com ara condicions “if-then” per classificar imatges o predir resultats, cosa que fallava amb dades complexes o incertes. En canvi, el machine learning permet que els ordinadors identifiquin patrons en conjunts de dades d'entrenament etiquetades i construeixin models predictius automàticament, com en la regressió lineal on s'aprèn la relació entre variables sense programar-la explícitament.

Aquest canvi s'ha accelerat gràcies a avanços en potència computacional, emmagatzematge de dades i algorismes com el descens de gradient, fent possible tractar problemes reals com el reconeixement d'imatges o la traducció automàtica.

0.1.1 Supervised/Unsupervised Learning

L'aprenentatge supervisat i no supervisat representen les dues branques principals del Machine Learning.

Supervised Learning (Aprenentatge Supervisat) En l'aprenentatge supervisat, el model s'entrena amb dades etiquetades que inclouen tant les entrades (característiques) com les sortides correctes (etiquetes), permetent aprendre la relació entre elles per fer prediccions en dades noves. Els tipus de problemes que poden resoldre són:

- **Regressió:** predir valors continus. Models: models de regressió lineals, polinòmica, etc.
- **Classificació:** assignar categories. Models: kNN, arbres de decisió, random forest, SVM.

Unsupervised Learning (Aprenentatge No Supervisat) L'aprenentatge no supervisat treballa amb dades sense etiquetes, descobrint patrons, estructures o agrupacions. Els tipus de problemes que poden resoldre són:

- **Segmentació (*Clustering*):** dividir clients, documents o imatges en grups per similituds. Models: k-means.

Introducció d'entrenament de models

Farem una introducció de termes clau a l'hora de parlar o entrenar models i ho farem a través d'un dels models més senzill: **la regressió lineal**.

El model **LinearRegression** pretén minimitzar l'ERROR QUADRÀTIC, així com en molts altres models de regressió.

###Exemple de predicció de l'evolució de la diabetes

Importarem les llibreries necessàries:

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_diabetes
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

###Llibreria Scikit-learn

Scikit-learn (o sklearn) és una de les llibreries més populars de Python per al Machine Learning. Ofereix eines senzilles i potents per fer classificació, regressió, clustering, selecció de variables i transformació de dades sense complicacions.

Recursos oficials:

* <https://scikit-learn.org/stable/>

Carregar les dades:

```
[2]: diabetes = load_diabetes()
df = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
df['target'] = diabetes.target
```

Paràmtre	Descripció
age	Edat en anys
sex	Gènere del pacient
bmi	Índex de massa corporal
bp	Pressió arterial mitjana
s1	Colesterol total sèric (tc)
s2	Lipoproteïnes de baixa densitat (ldl)
s3	Lipoproteïnes d'alta densitat (hdl)
s4	Colesterol total / HDL (tch)
s5	Nivell de triglicèrids (ltg)
s6	Nivell de glucosa en sang

Abans de tot heu de saber les característiques de les dades que tenim i per tant hem de realitzar un **Anàlisis Exploratori de les Dades (EDA)**.

- Entendre estructura (tipus dades, missing values)
- Detectar anomalies (outliers, duplicats)
- Trobar patrons (correlacions, distribucions)
- Visualitzar relacions (variables vs target)
- Preparar dades (feature engineering)

```
[3]: df.head()
```

```
[3]:      age      sex      bmi      bp      s1      s2      s3 \
0  0.038076  0.050680  0.061696  0.021872 -0.044223 -0.034821 -0.043401
1 -0.001882 -0.044642 -0.051474 -0.026328 -0.008449 -0.019163  0.074412
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -0.032356
3 -0.089063 -0.044642 -0.011595 -0.036656  0.012191  0.024991 -0.036038
4  0.005383 -0.044642 -0.036385  0.021872  0.003935  0.015596  0.008142

      s4      s5      s6  target
0 -0.002592  0.019907 -0.017646   151.0
1 -0.039493 -0.068332 -0.092204    75.0
2 -0.002592  0.002861 -0.025930   141.0
3  0.034309  0.022688 -0.009362   206.0
4 -0.002592 -0.031988 -0.046641   135.0
```

```
[4]: df.shape
```

```
[4]: (442, 11)
```

```
[5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 442 entries, 0 to 441
Data columns (total 11 columns):
#   Column  Non-Null Count  Dtype
---  -
0    age    442 non-null      float64
1    sex    442 non-null      float64
2    bmi    442 non-null      float64
3    bp     442 non-null      float64
4    s1     442 non-null      float64
5    s2     442 non-null      float64
6    s3     442 non-null      float64
7    s4     442 non-null      float64
8    s5     442 non-null      float64
9    s6     442 non-null      float64
10   target  442 non-null      float64
dtypes: float64(11)
memory usage: 38.1 KB
```

```
[6]: df.describe()
```

```
[6]:      age      sex      bmi      bp      s1 \
count  4.420000e+02  4.420000e+02  4.420000e+02  4.420000e+02  4.420000e+02
mean   -2.511817e-19  1.230790e-17 -2.245564e-16 -4.797570e-17 -1.381499e-17
std     4.761905e-02  4.761905e-02  4.761905e-02  4.761905e-02  4.761905e-02
min    -1.072256e-01 -4.464164e-02 -9.027530e-02 -1.123988e-01 -1.267807e-01
```

25%	-3.729927e-02	-4.464164e-02	-3.422907e-02	-3.665608e-02	-3.424784e-02
50%	5.383060e-03	-4.464164e-02	-7.283766e-03	-5.670422e-03	-4.320866e-03
75%	3.807591e-02	5.068012e-02	3.124802e-02	3.564379e-02	2.835801e-02
max	1.107267e-01	5.068012e-02	1.705552e-01	1.320436e-01	1.539137e-01

	s2	s3	s4	s5	s6 \
count	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02
mean	3.918434e-17	-5.777179e-18	-9.042540e-18	9.293722e-17	1.130318e-17
std	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02
min	-1.156131e-01	-1.023071e-01	-7.639450e-02	-1.260971e-01	-1.377672e-01
25%	-3.035840e-02	-3.511716e-02	-3.949338e-02	-3.324559e-02	-3.317903e-02
50%	-3.819065e-03	-6.584468e-03	-2.592262e-03	-1.947171e-03	-1.077698e-03
75%	2.984439e-02	2.931150e-02	3.430886e-02	3.243232e-02	2.791705e-02
max	1.987880e-01	1.811791e-01	1.852344e-01	1.335973e-01	1.356118e-01

	target
count	442.000000
mean	152.133484
std	77.093005
min	25.000000
25%	87.000000
50%	140.500000
75%	211.500000
max	346.000000

```
[7]: df.isnull().sum()
```

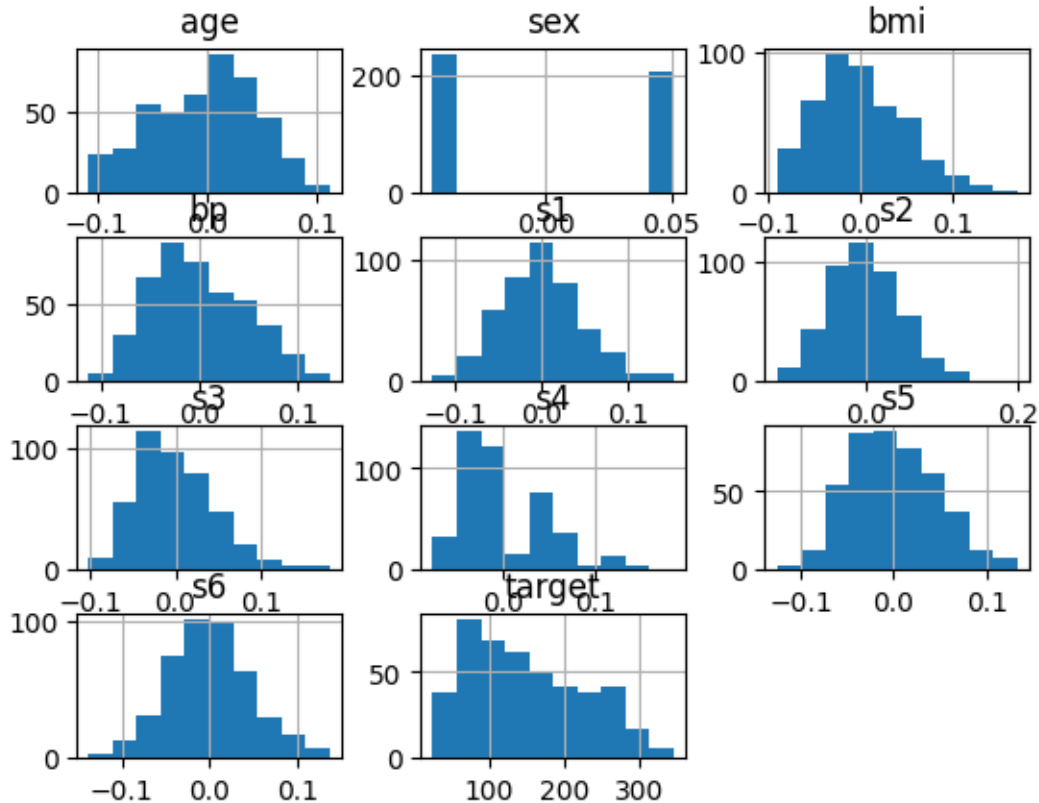
```
[7]: age      0
sex      0
bmi      0
bp      0
s1      0
s2      0
s3      0
s4      0
s5      0
s6      0
target   0
dtype: int64
```

```
[8]: df.duplicated().sum()
```

```
[8]: np.int64(0)
```

```
[9]: df.hist()
```

```
[9]: array([[<Axes: title={'center': 'age'}>, <Axes: title={'center': 'sex'}>,
<Axes: title={'center': 'bmi'}>],
[<Axes: title={'center': 'bp'}>, <Axes: title={'center': 's1'}>,
<Axes: title={'center': 's2'}>],
[<Axes: title={'center': 's3'}>, <Axes: title={'center': 's4'}>,
<Axes: title={'center': 's5'}>],
[<Axes: title={'center': 's6'}>,
<Axes: title={'center': 'target'}>, <Axes: >]], dtype=object)
```



###Entrenament del model

L'entrenament és optimització matemàtica per tal de buscar els millors paràmetres que minimitzin l'error entre prediccions del model i valors reals.

```
[10]: X1 = df[['bmi']]
y1 = df['target']

model1 = LinearRegression()
model1.fit(X1, y1)
y1_pred = model1.predict(X1)

print(f"Fórmula: y = {model1.intercept_:.0f} + {model1.coef_[0]:.0f}x")
```

Fórmula: $y = 152 + 949x$

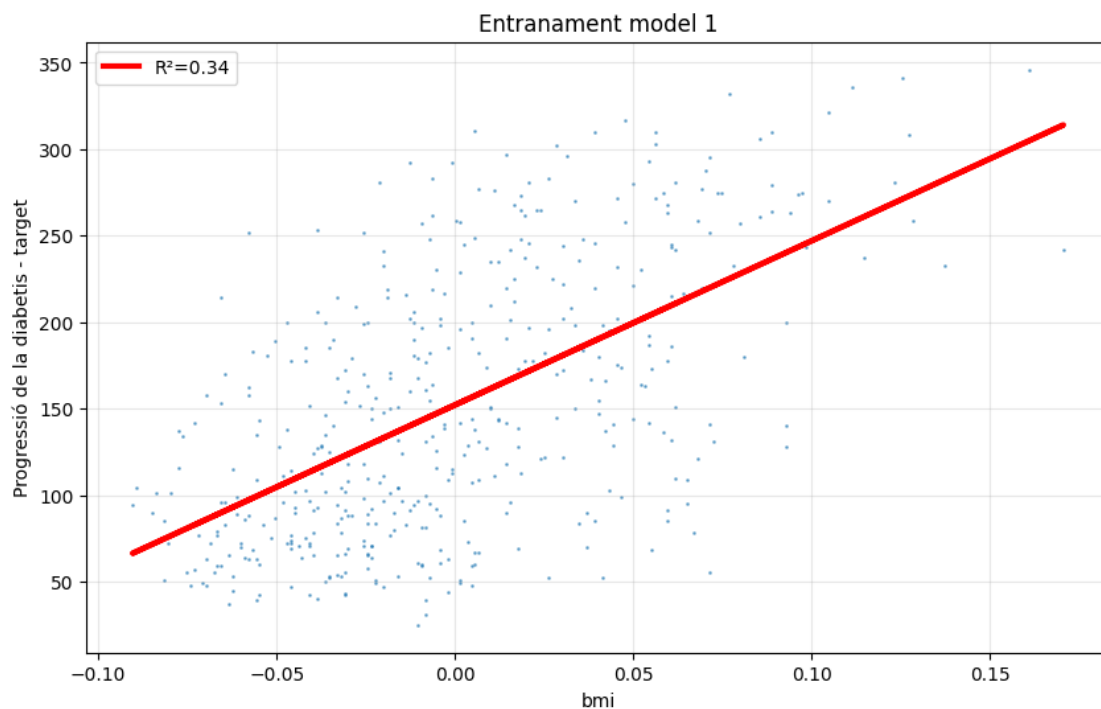
```
[11]: # Càlcul d'errors i mètriques d'efectivitat
import numpy as np
r2 = r2_score(y1, y1_pred)
mse = mean_squared_error(y1, y1_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y1, y1_pred)
print(f"RMSE (root mean square error):{rmse:.2f}")
print(f"R² TOTAL: {r2:.4f}")
print(f"MAE (mean absolut error):{mae:.2f}")
```

RMSE (root mean square error):62.37

R² TOTAL: 0.3439

MAE (mean absolut error):51.80

```
[12]: plt.figure(figsize=(10,6))
plt.scatter(X1, y1, alpha=0.5, s=1)
plt.plot(X1, y1_pred, 'r-', linewidth=3, label=f'R²={r2:.2f}')
plt.xlabel('bmi'); plt.ylabel('Progressió de la diabetis - target')
plt.title('Entranament model 1')
plt.legend(); plt.grid(True, alpha=0.3)
plt.show()
```



PROBLEMA Estem avaluant sobre les mateixes dades l'entrenament. * El model ja ha vist totes

les dades. * Les mètriques són massa optimistes. * No sabem com funcionarà amb dades noves. * Risc d'OVERFITTING (sobreajustament).

Com podem validar les dades i millorar el model? Necessitem unes dades de TRAINING i unes de TEST

```
[13]: from sklearn.model_selection import train_test_split

X1 = df[['bmi']]
y1 = df['target']

# Dividir dades
X_train, X_test, y_train, y_test = train_test_split(X1, y1, test_size=0.2,
                                                    random_state=42)

model1 = LinearRegression()
model1.fit(X_train, y_train)

# Predir sobre dades de TEST (que el model NO ha vist)
y_pred_test = model1.predict(X_test)
y_pred_train = model1.predict(X_train)
```

```
[14]: print(f"\nDades d'entrenament: {len(y_train)} pacients (80%)")
      print(f"Dades de test: {len(y_test)} pacients (20%)")
```

Dades d'entrenament: 353 pacients (80%)

Dades de test: 89 pacients (20%)

Calculem les mètriques de d'ENTRENAMENT i de TEST

```
[15]: # Calcular mètriques en ENTRENAMENT
r2_train = r2_score(y_train, y_pred_train)
rmse_train = np.sqrt(mean_squared_error(y_train, y_pred_train))
mae_train = mean_absolute_error(y_train, y_pred_train)

# Calcular mètriques en TEST
r2_test = r2_score(y_test, y_pred_test)
rmse_test = np.sqrt(mean_squared_error(y_test, y_pred_test))
mae_test = mean_absolute_error(y_test, y_pred_test)
```

```
[16]: print(f"{'R²':<15} {r2_train:<20.4f} {r2_test:<20.4f}")
      print(f"{'RMSE':<15} {rmse_train:<20.2f} {rmse_test:<20.2f}")
      print(f"{'MAE':<15} {mae_train:<20.2f} {mae_test:<20.2f}")
```

R ²	0.3657	0.2334
RMSE	62.08	63.73
MAE	51.38	52.26

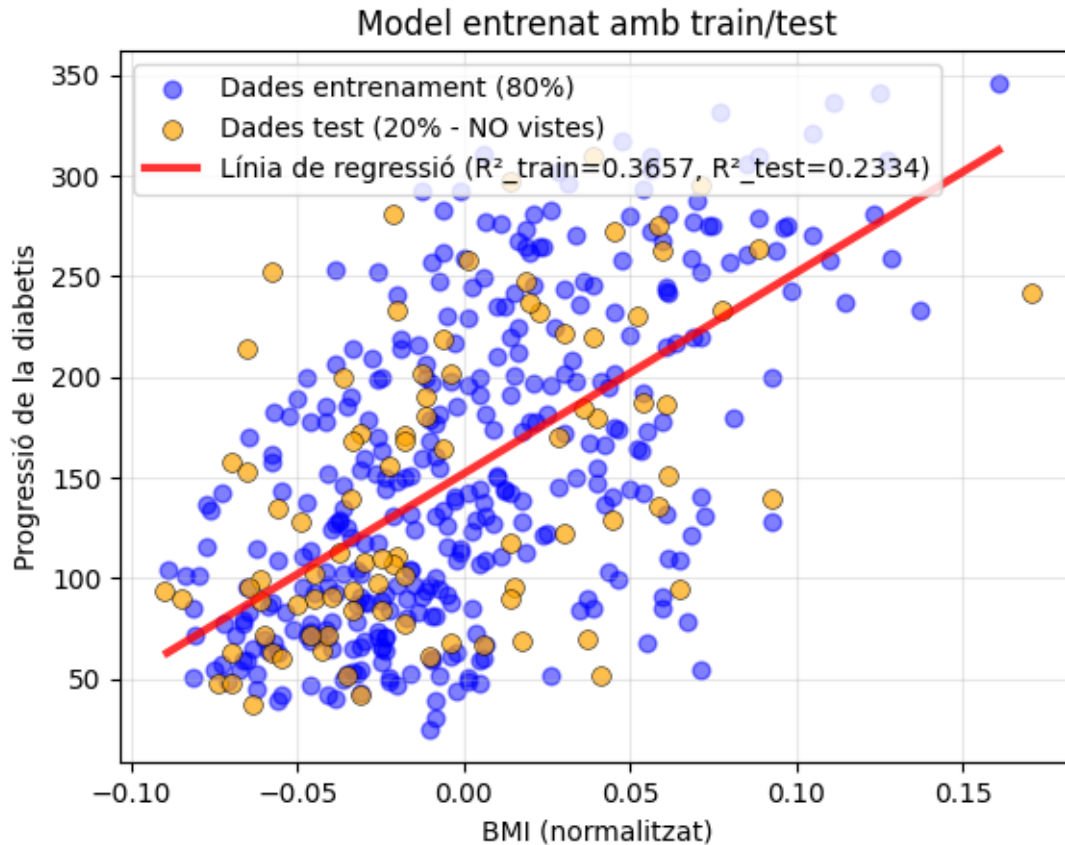
```
[17]: # Combinar train i test en un sol gràfic
plt.scatter(X_train, y_train, alpha=0.5, label='Dades entrenament (80%)',
            color='blue', s=40)
plt.scatter(X_test, y_test, alpha=0.7, label='Dades test (20% - NO vistes)',
            color='orange', s=50, edgecolors='black', linewidth=0.5)

# Ordenar per visualitzar millor la línia
X_train_sorted = np.sort(X_train, axis=0)
y_pred_train_sorted = model1.predict(X_train_sorted)

plt.plot(X_train_sorted, y_pred_train_sorted, color='red', linewidth=3,
         label=f'Línia de regressió ( $R^2_{\text{train}}={r2\_train:.4f}$ ,  $R^2_{\text{test}}={r2\_test:.4f}$ '),
         alpha=0.8)

plt.xlabel('BMI (normalitzat)')
plt.ylabel('Progressió de la diabetis')
plt.title(f'Model entrenat amb train/test')
plt.legend(loc='best', fontsize=10)
plt.grid(True, alpha=0.3)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
  warnings.warn(
```

Analitzem la diferència entre train i test:

```
[18]: # Analitzar diferència entre train i test
diff_r2 = abs(r2_train - r2_test)
print(f"\nDiferència R² (train - test): {diff_r2:.4f}")
if diff_r2 < 0.05:
    print("Molt poca diferència : Model generalitza BÉ")
elif diff_r2 < 0.15:
    print("Diferència acceptable : Model generalitza bé")
else:
    print("Gran diferència : Possible OVERFITTING")
```

Diferència R^2 (train - test): 0.1324

Diferència acceptable : Model generalitza bé

Alguns aspectes importants a tenir en compte: * SEMPRE divideix les dades en train i test * Les mètriques de TEST són les que realment ens importen * Gran diferència entre train-test indica OVERFITTING.

```
[19]: import seaborn as sns
```

```
dades = sns.load_dataset('diamonds')
dades
```

```
[19]:
```

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...	...
53935	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

```
[53940 rows x 10 columns]
```

```
[20]: dades.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53940 entries, 0 to 53939
Data columns (total 10 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   carat       53940 non-null  float64
 1   cut         53940 non-null  category
 2   color       53940 non-null  category
 3   clarity     53940 non-null  category
 4   depth       53940 non-null  float64
 5   table       53940 non-null  float64
 6   price       53940 non-null  int64
 7   x           53940 non-null  float64
 8   y           53940 non-null  float64
 9   z           53940 non-null  float64
dtypes: category(3), float64(6), int64(1)
memory usage: 3.0 MB
```

```
[21]: dades.isnull().sum()
```

```
[21]: carat      0
      cut       0
      color    0
      clarity  0
      depth    0
```

```

table      0
price      0
x          0
y          0
z          0
dtype: int64

```

```
[22]: dades.describe()
```

```

[22]:
      count  carat      depth      table      price      x \
count  53940.000000  53940.000000  53940.000000  53940.000000  53940.000000
mean    0.797940    61.749405    57.457184    3932.799722    5.731157
std     0.474011     1.432621     2.234491    3989.439738     1.121761
min     0.200000    43.000000    43.000000    326.000000     0.000000
25%     0.400000    61.000000    56.000000    950.000000     4.710000
50%     0.700000    61.800000    57.000000   2401.000000     5.700000
75%     1.040000    62.500000    59.000000   5324.250000     6.540000
max      5.010000    79.000000    95.000000  18823.000000    10.740000

      y      z
count  53940.000000  53940.000000
mean    5.734526     3.538734
std     1.142135     0.705699
min     0.000000     0.000000
25%     4.720000     2.910000
50%     5.710000     3.530000
75%     6.540000     4.040000
max     58.900000    31.800000

```

```

[23]: files, columnnes = dades.shape
print(f"Files: {files} - Columnnes: {columnnes}")

```

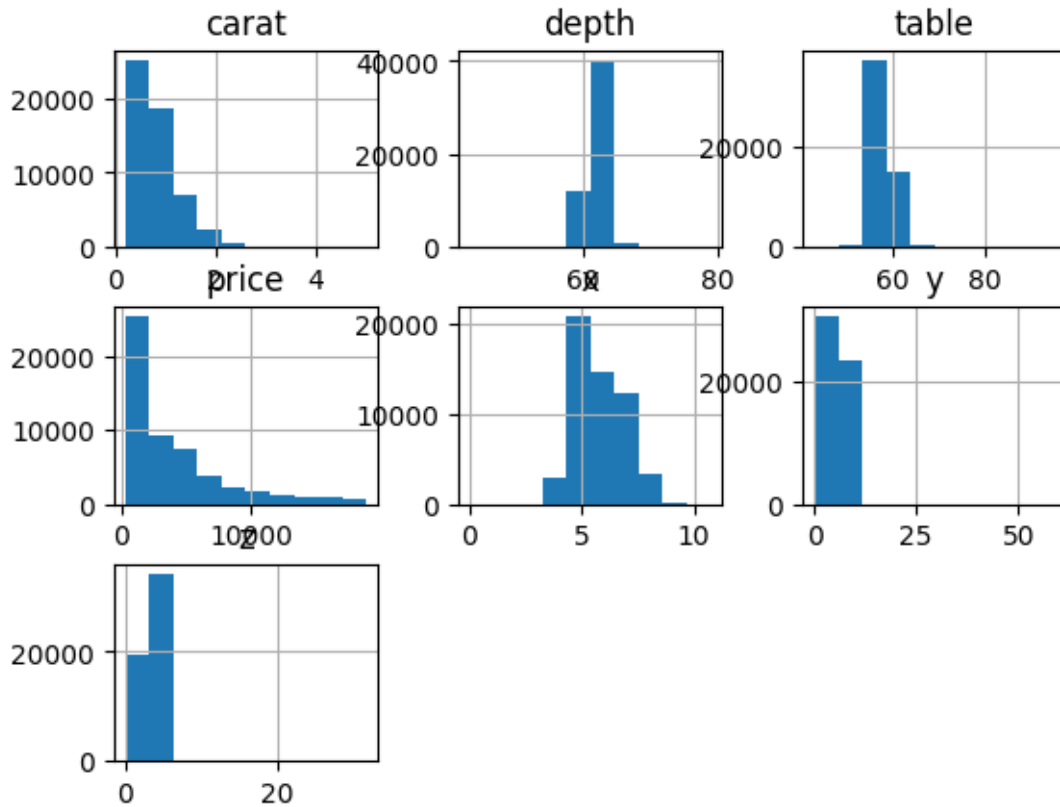
```
Files: 53940 - Columnnes: 10
```

```
[24]: dades.hist()
```

```

[24]: array([[<Axes: title={'center': 'carat'}>,
      <Axes: title={'center': 'depth'}>,
      <Axes: title={'center': 'table'}>],
      [<Axes: title={'center': 'price'}>, <Axes: title={'center': 'x'}>,
      <Axes: title={'center': 'y'}>],
      [<Axes: title={'center': 'z'}>, <Axes: >, <Axes: >]], dtype=object)

```



```
[25]: daes.duplicated().sum()
```

```
[25]: np.int64(146)
```

```
[26]: daes.drop_duplicates(inplace=True)
daes.duplicated().sum()
```

```
[26]: np.int64(0)
```

```
[27]: daes.reset_index(drop=True)
```

```
[27]:
```

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...	...
53789	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53790	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53791	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56

53792	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53793	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

[53794 rows x 10 columns]

```
[28]: X1 = dados[['carat']]
      y1 = dados['price']

      model1 = LinearRegression()
      model1.fit(X1, y1)
      y1_pred = model1.predict(X1)

      print(f"Fórmula: y = {model1.intercept_:.0f} + {model1.coef_[0]:.0f}x")
```

Fórmula: y = -2261 + 7764x